

General Instructions:

- This is a time-bound (2 hours), open-book, closed-Internet, no-LLM, and **individual** test. You may visit **ONLY** the white-listed websites (within ExamenA) during the test.
- You must test your web pages using **Google Chrome Web Browser** (Version 139.x or later). Your graders will be using only **Google Chrome Web Browser** (Version 139.x) to test your web pages.
- **No questions will be entertained** by the IS216 teaching team or the invigilators during the test period. If necessary, make your own assumptions and proceed to complete test questions.
- You must **use only standard HTML5, CSS, Bootstrap (Version 5.3), JavaScript and Axios** in your solutions unless the question specifies otherwise. Do not use any other third-party libraries (e.g. Angular, React, or others).
- Unnecessary use of v-html is prohibited. Any unnecessary use of v-html found in your code will result in a penalty of 1 mark per occurrence.
- Use meaningful names for HTML class/id and JavaScript variables and functions. You must indent your code (HTML/CSS/JavaScript) properly. Failure to do so will attract a penalty of up to **20%** of your score for the corresponding question.
- You **MUST** include your name as author in the comments of all your submitted source files. Failure to do so will attract a penalty of up to **20%** of your score for the corresponding question. For example, if your registered name is "KIM Jong Un" and email ID is kim.jongun.2024, include the following comment at the beginning of each source file you write.

Vue & CSS Files
<pre>/* Name: Kim Ju Ae Email: kim.juae.2026 */</pre>

- You may wish to comment out the parts in your code which cause errors. Commented code will not be marked.

Academic Integrity

- All student submissions will be thoroughly checked by an SMU-approved source code plagiarism checker software program AND an additional external software program. The source code checking will be conducted across all submissions from all sections of IS216.
- Suspected plagiarism cases will be reported immediately to the IS216 faculty in charge and SCIS Dean's Office for further investigation.
- Students in the suspected cases will be informed accordingly by their section faculty, and the incident will be escalated to the SMU University Council of Student Conduct.

Submission Instructions

- **Due Date**

- **XXXX**
 - This deadline **may vary** for students with time extensions (*please check with your invigilator for details*).
- Late submission policy is as follows:

Submit within 5 minutes of set deadline	10% penalty of your entire test's score
Beyond 5 minutes	0 mark

- **Zip** up all files in **Q1/Q2/Q3/Q4** folders into **<YOUR_SMU_ID>.zip**
 - For example, **kim.juae.2026.zip**
 - **Verify by unzipping this zip file – check the content inside**
 - **Incorrect submission file name** WILL attract a penalty of up to **20%** of your score for the entire test.
- Only **zip** format is accepted.
 - **.7z, rar** or other **compression formats** are **NOT** accepted.
 - Until the correct **zip** format is submitted again by the student, it will be assumed that the student has NOT made the submission and late submission policy will apply.
- Submit the **zip** file in **Examina**:
 - **It is your (student's) individual responsibility to ensure that the zip file submission was successful.**
 - **Your section faculty, instructor, and Teaching Assistants will NOT verify the submission for you.**

Legend



Do NOT edit this given resource file.



Your answer/code goes here into this given resource file.

Q1. CSS

[5 Marks]

Given resources in folder Q1

 q1.vue

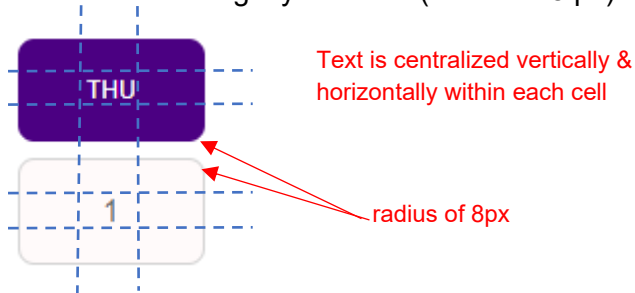
 q1.css

Instructions

1. You are to style the HTML elements using **vanilla CSS (no Bootstrap)**.
2. Open *q1.vue* in a browser and examine how it looks.
3. You are to edit only *q1.css file*.

Part A (2 marks)

Edit the CSS file so that each calendar cell's text is centralized both horizontally and vertically. The border should also be slightly rounded (radius of 8 px) instead of rectangular:



This is how the calendar should look after the change:

October 2026						
SUN	MON	TUE	WED	THU	FRI	SAT
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

October 2026						
SUN	MON	TUE	WED	THU	FRI	SAT
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Part B (2 marks)

In line 30 of *q1.vue*, "2nd Oct 2026" has been denoted "today". Edit the CSS file so that the calendar cell showing today looks different:

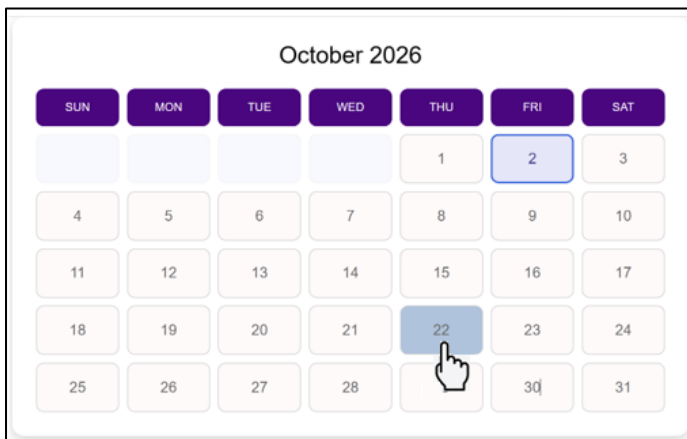
- Background color = lavender
- Text color = darkslateblue
- Border is 2 pixels thick and royalblue in color

October 2026						
SUN	MON	TUE	WED	THU	FRI	SAT
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

today

Part C (1 mark)

Edit the CSS so that when the mouse hovers a cell, the background color of that the (i) cell turns to lightsteelblue, and (ii) the cursor changes to a pointer (hand icon). The screenshot below shows when the mouse moves over the "22nd Oct" cell:



(Hint: check out the "**cursor**" property for the **:hover** selector)

Q2. Bootstrap

[5 Marks]

Given resources in folder `client/src/components/q2`

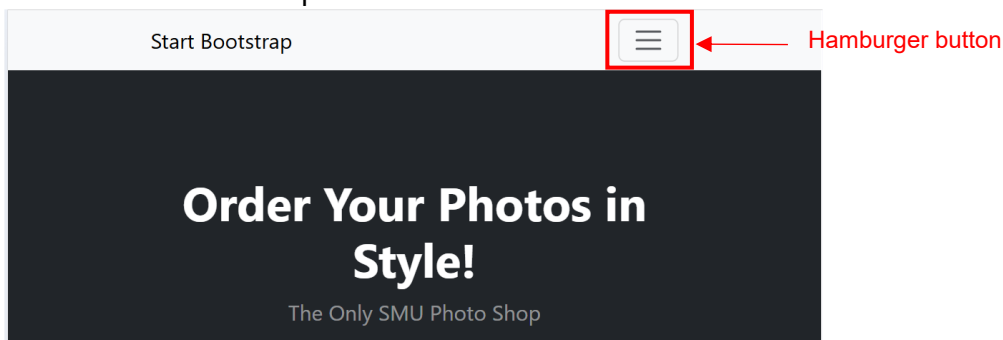
 `q2.vue`

Instructions

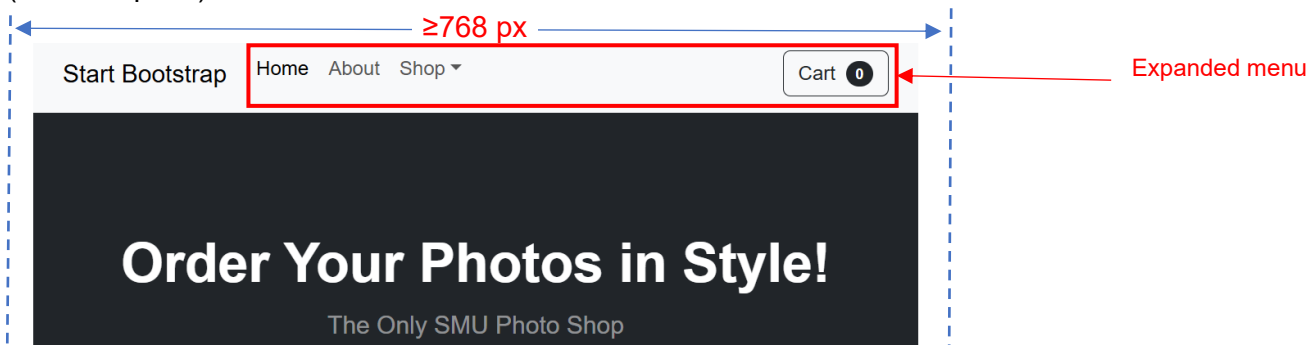
1. You will **edit** `q2.vue`.
2. You are to solve the web page layout problem by using **Bootstrap 5.3** grid, utilities, and breakpoints.
3. Do not add custom CSS or external libraries.
4. Do not change IDs or provided text.
5. You are not required to add/edit other styles, such as margin, padding, font style, font size, etc., not mentioned in the requirements.

Part A (1 mark)

Currently the navigation bar at the top appears as a hamburger button at the upper right corner regardless of the width of the viewport:



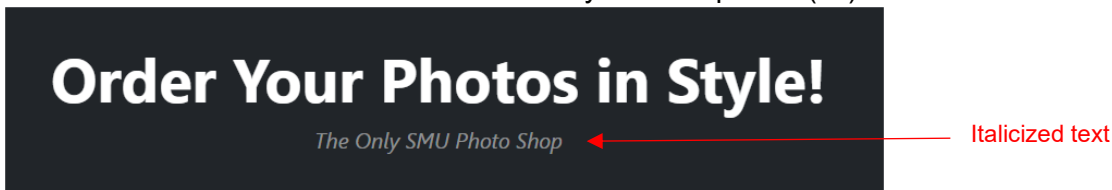
Edit the file so that the navigation bar expands (appears in full) when the width of the view port is ≥ 768 px (**md** breakpoint):



However, if the width is smaller than the **md** breakpoint, the navigation bar appears as a hamburger button as before.

Part B (1 mark)

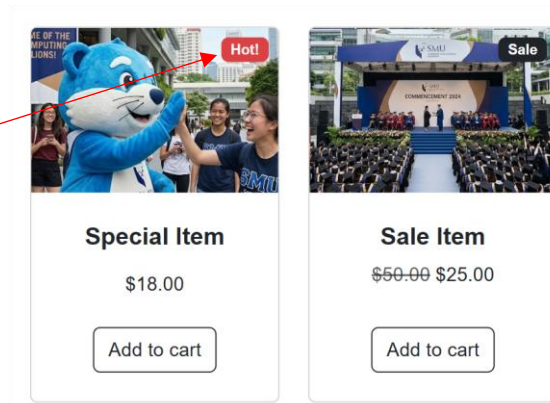
Italicize the subtitle as shown below. Use only Bootstrap class(es) to achieve this.



Part C (1 mark)

Edit the "Hot!" badges so that they look red (with white text). "Sale" badges should remain black (with white text).

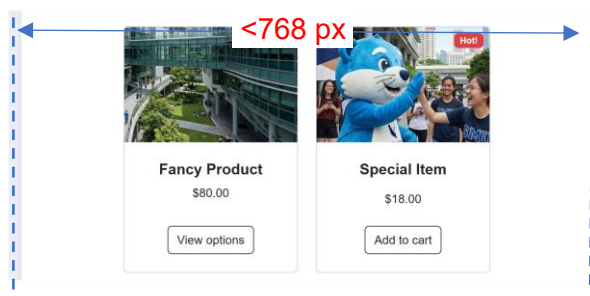
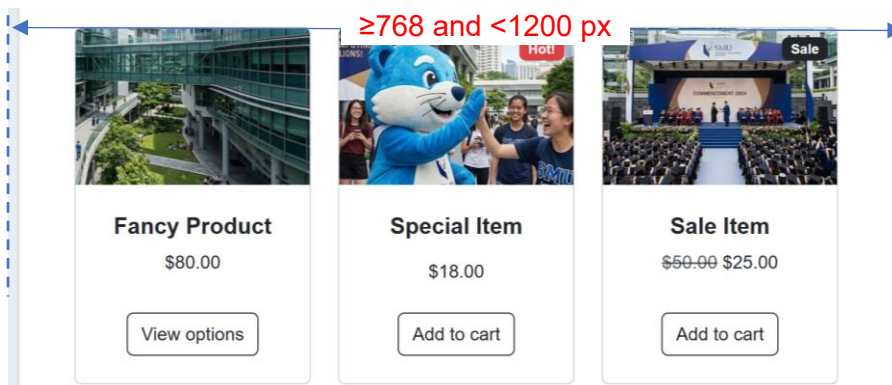
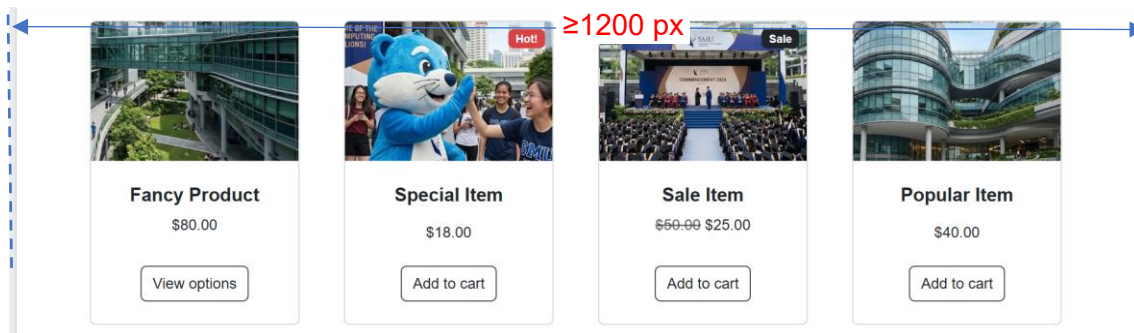
"Hot!" badges should have a red background.



Part D (2 marks)

Currently, the 8 product cells always appear in 3 equal-width columns regardless of the viewport's width. Edit the file so that:

- when the browser width is ≥ 1200 px (**x1** breakpoint), the cells display in 4 equal-width columns, and
- when the browser width is ≥ 768 px (**md** breakpoint) but < 1200 px, the cells display in 3 equal-width columns, and
- when the browser width is < 768 px (**md** breakpoint), the cells display in 2 equal-width columns.



Q3. Basic Vue

[5 Marks]

Given resources in folder q3

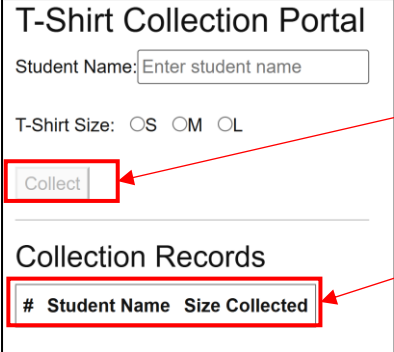
 q3.vue
 data.js

Instructions

1. You are to edit only q3.vue.

Original Output

When q3.vue component is loaded on the browser for the first time, it will show the following output:



T-Shirt Collection Portal

Student Name:

T-Shirt Size: ☐ S ☐ M ☐ L

Collection Records

#	Student Name	Size Collected
---	--------------	----------------

Annotations: Red arrows point to the 'Collect' button with the text 'Disabled until name filled in and size selected' and to the table header with the text 'Table header appears'.

This Web page is used by staff when students come to collect their T-shirts. The staff enters the student's name and T-shirt size, then clicks on 'Collect' before handing the shirt over. For now:

- The 'Collect' button is enabled only when (i) the name field is filled and (ii) one of the 3 radio buttons has been selected.
- The table header appears even when there is no collection record.
- When a collection happens, the table currently does not show any new data.

Part A (1 mark)

Edit q3.vue such that the quantity of T-shirts for each size is shown in brackets behind each radio button. Since each quantity is initialized to 5 in the <script> part, the page should show this at the beginning:



T-Shirt Size: ☐ S (5) ☐ M (5) ☐ L (5)

Quantity left for each size

When the Collect button is clicked, the quantity for the relevant size decreases by 1. For example, after 4 M-sized and 2 L-sized shirts have been collected, the page should show this:



T-Shirt Size: ☐ S (5) ☐ M (1) ☐ L (3)

Part B (1 mark)

When the quantity of a particular size drops to 0, that radio button should be disabled so that it cannot be selected:



T-Shirt Size: ☐ S (5) ☐ M (0) ☐ L (3)

Disabled radio button

Part C (1 mark)

Edit q3.vue so that if no collection has happened, the message "No collection records yet" appears instead of the table header. The table header only appears when there is at least 1 collection record.

T-Shirt Size: ☐ S (5) ☐ M (5) ☐ L (5)

Collection Records

No collection records yet.

Appears (instead of the table header) when nobody has collected

Part D (2 marks)

Every time a collection happens, a new row is shown with the entered student name and selected size. For example, the page can look like this after 9 shirts have been collected:

T-Shirt Collection Portal

Student Name:

T-Shirt Size: ☐ S (1) ☐ M (2) ☐ L (3)

Collection Records

#	Student Name	Size Collected
1	Lay Foo	M
2	Cordelia	M
3	Chein	L
4	Alvin	S
5	Mokkie	L
6	Shar	M
7	Kyong	S
8	Keith	S
9	Jason	S

1 row for each collection

The names are inserted in order of collection. The remaining quantity of each size should tally with the sizes of the shirts collected.

Q4. Vue – AXIOS & JSON

[10 Marks]

Given `client` resources in folder **src/components/q4**

 `q4.vue`

 `q4_readonly.css`

Note: You are also given the `server` resources in folder **resources/server**, which you need for this question.

Instructions

1. Edit only `q4.vue`.
2. Run server in **resources/server** using `pnpm install`, followed by `pnpm dev`. There is **NO** need to change anything in the server folder.
2. Use **Axios** for all HTTP calls.
4. No external libraries except Axios are allowed. Do not add custom CSS.
5. Do **NOT** hardcode.

Scenario & API Documentation

The CourseInfo server (hosted at `127.0.0.1/api`) hosts a web service to retrieve all course information, add new courses and delete existing courses. Your Vue page will send async HTTP requests to perform these 3 functions. See the last two pages for examples of GET, POST and DELETE requests using **Postman**.

Part A: `loadData` (2 marks)

Currently `q4.vue` looks like this:

Add New Course

Course Code:

Course Name:

Description:

☐ IS Core ☐ SE Core ☐ CS Core ☐ CL Core

Add Course

Code	Name	Description	IS Core	SE Core	CS Core	CL Core	Actions
------	------	-------------	---------	---------	---------	---------	---------

If you see "**Failed to load course data**" at the top of the page, it is likely that your CourseInfo Server is not running yet.

`loadData()` is called upon mounting, but `loadData` currently doesn't do much.

(1) Edit **loadData()** so that it makes an axios GET request to the CourseInfo Server and assign the returned JSON object to **courses**. If an error occurs, update **errorMessage** to show "Failed to load course data". Once **courses** has updated correctly, information about each course will display in the table on the page, like this:

Add New Course

Course Code:

Course Name:

Description:

☐ IS Core
 ☐ SE Core
 ☐ CS Core
 ☐ CL Core

Code	Name	Description	IS Core	SE Core	CS Core	CL Core	Actions
is113	Web Application Development I	Vibe coding using JavaScript					Delete
cs101	Programming I	Vibe coding using C					Delete
se201	AI Driven Software Engineering	How to be artificially intelligent					Delete
se301	Advanced Programming and Design	Advanced vibe coding					Delete

Part B: addCourse (3 marks)

Fill the 3 text fields (course code, course name and description) with sample information and click on "Add Course". Nothing happens for now.

- (1) Edit the Vue file such that when the "Add Course" button is clicked, the **addCourse()** function will run
- (2) Fill up the **addCourse()** function to make an axios POST request to the CourseInfo Server with the correct payload extracted from the "Add New Course" section at the top of the page. If an error occurs, update **errorMessage** to show "Failed to add course". Declare and use additional reactive constants if you need to.
- (3) Upon a successful POST, the table should display the newly added course at the last row.
- (4) **addCourse()** should also clear all fields and checkboxes in the "Add Course" form at the top upon a successful POST so that it is ready for the next course addition.

Part C: deleteCourse (2 marks)

- (1) When a Delete button for a particular row is clicked, it will call the **deleteCourse** function and pass over the course code. Insert code into **deleteCourse()** to make an axios DELETE request to the CourseInfo Server with the course code in the URL. If an error occurs, update **errorMessage** to show "Failed to delete course".
- (2) Upon a successful DELETE, the table should not show the recently deleted row.

Part D: Yes and No (3 marks)

(1) Edit the Vue file so that "Yes" (for true) and "No" (for false) will appear for each course. Use the **"text-yes"** and **"text-no"** classes (defined in **q4_readonly.css**), so that when the styles in these classes (in **q4_readyonly.css**) are replaced during marking, the changes will be automatically reflected in the table.

The table should look like this:

Add New Course

Course Code:
e.g. is113

Course Name:
e.g. Web App Dev

Description:
Course details...

☐ IS Core ☐ SE Core ☐ CS Core ☐ CL Core

Add Course

Code	Name	Description	IS Core	SE Core	CS Core	CL Core	Actions
is113	Web Application Development I	Vibe coding using JavaScript	Yes	Yes	No	Yes	Delete
cs101	Programming I	Vibe coding using C	No	No	Yes	No	Delete
se201	AI Driven Software Engineering	How to be artificially intelligent	Yes	Yes	No	No	Delete
se301	Advanced Programming and Design	Advanced vibe coding	No	Yes	No	No	Delete

Example of **GET** call using **Postman**

Retrieving all courses

GET ⌵ http://127.0.0.1:8000/api

Params Authorization Headers (8) Body • Pre-request Script Tests Settings

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON ⌵

```
1 {  
2   "courseInfo": [  
3     {  
4       "code": "is113",  
5       "name": "Web Application Development I",  
6       "description": "The backend is your playground!",  
7       "coreForIS": true,  
8       "coreForSE": true,  
9       "coreForCS": false,  
10      "coreForCL": true  
11    },  
12    {  
13      "code": "cs101",  
14      "name": "Programming I",  
15      "description": "How to vibe code using C",  
16      "coreForIS": false,  
17      "coreForSE": false,  
18      "coreForCS": true,  
19      "coreForCL": false  
20    },  
21    {  
22      "code": "se201",
```

Example of **POST** call using **Postman**

Adding a new course with code "cs102"

POST ⌵ http://127.0.0.1:8000/api

Params Authorization Headers (8) Body • Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON ⌵

```
1 {  
2   "code": "cs102",  
3   "name": "Programming II",  
4   "description": "Vibe Coding using Java",  
5   "coreForIS": false,  
6   "coreForSE": true,  
7   "coreForCS": true,  
8   "coreForCL": false  
9 }
```

Body Cookies Headers (10) Test Results

Pretty Raw Preview Visualize JSON ⌵

```
3   "item": {  
4     "code": "cs102",  
5     "name": "Programming II",  
6     "description": "Vibe Coding using Java",  
7     "coreForIS": false,  
8     "coreForSE": true,  
9     "coreForCS": true,  
10    "coreForCL": false  
11  }  
12 }
```

An error will be returned if the course code already exists

POST http://127.0.0.1:8000/api

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "code": "cs102",
3   "name": "Testing II",
4   "description": "Sample course",
5   "coreForIS": false,
6   "coreForSE": true,
7   "coreForCS": true,
8   "coreForCL": false
9 }
```

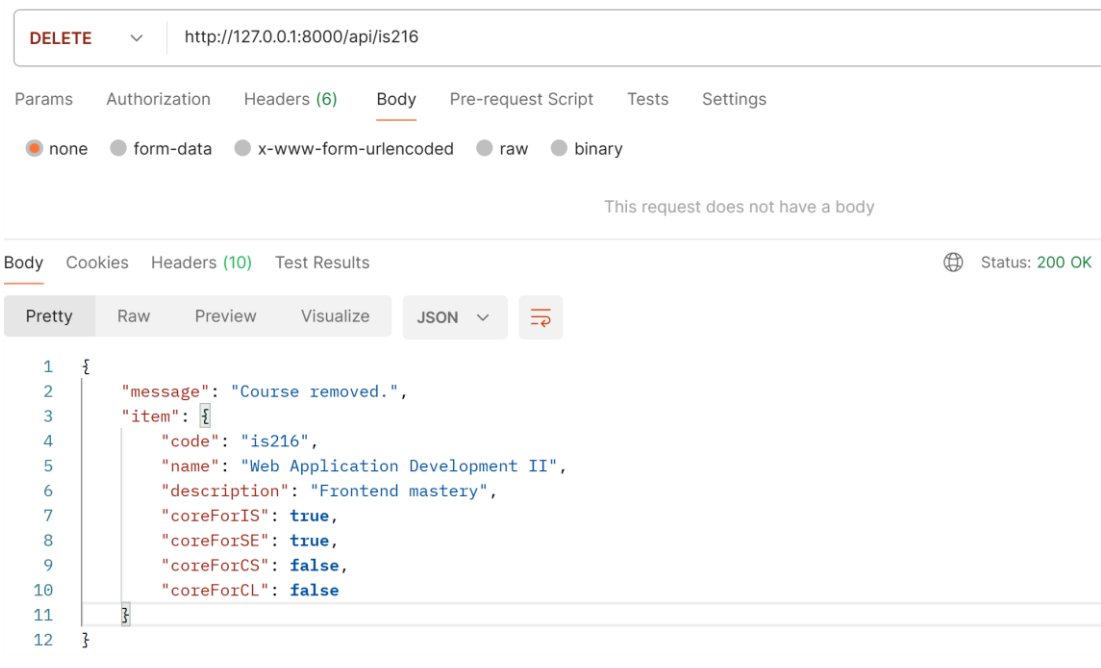
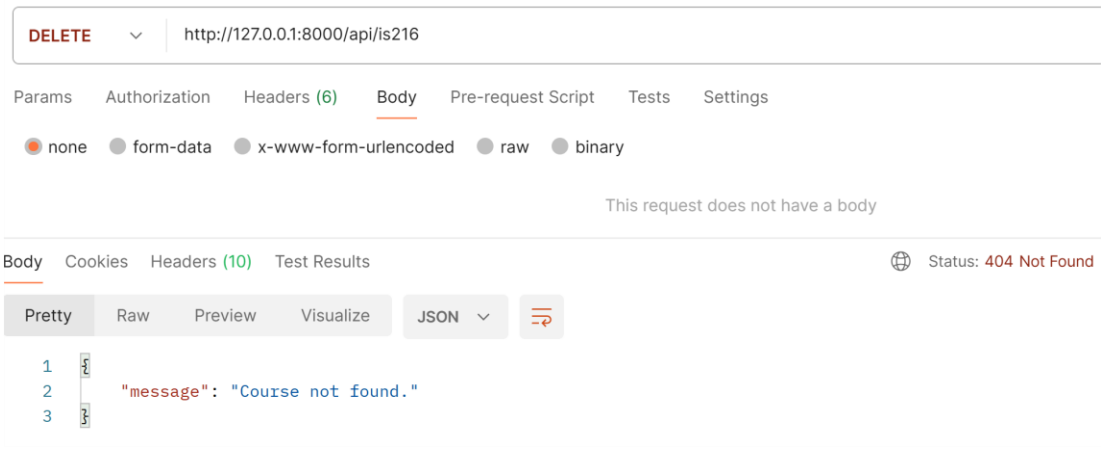
Body Cookies Headers (10) Test Results

Status: 409 Conflict

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Course code already exists."
3 }
```

Example of **DELETE** call using **Postman**

Deleting a course with code "is216"	 The screenshot shows a Postman interface for a DELETE request to <code>http://127.0.0.1:8000/api/is216</code>. The request is successful, returning a 200 OK status. The response body is a JSON object: <pre>{ "message": "Course removed.", "item": { "code": "is216", "name": "Web Application Development II", "description": "Frontend mastery", "coreForIS": true, "coreForSE": true, "coreForCS": false, "coreForCL": false } }</pre>
An error will be returned if the course code does not exist	 The screenshot shows a Postman interface for a DELETE request to <code>http://127.0.0.1:8000/api/is216</code>. The request fails, returning a 404 Not Found status. The response body is a JSON object: <pre>{ "message": "Course not found." }</pre>

END